Throttling vs Rate Limiting vs Quotas

free and paid tiers, monthly limits, and fairness

Watch the 3-min video

> **In one line:** rate limiting rejects requests above a short-term rate with 429, throttling slows or queues them instead, load shedding drops low-priority work when the server is overloaded, and quotas are long-term per-plan budgets that need durable, billing-grade counting.

1 Why a per-second limit isn't enough

Your API has a **free tier** with 1,000 requests per day and a **paid tier** with 1,000,000 per month. A free user writes a loop and burns the whole day's allowance in ten minutes. A paying customer runs a giant batch at month end. A per-second rate limit catches neither problem on its own. You need three tools: **throttling**, **quotas** and **tiers**.

2 The words: rate limiting, throttling, load shedding

- **Rate limiting** enforces a short-term rate, like 100 per minute, and **rejects** the excess with `429 Too Many Requests`.
- **Throttling** usually means **slowing down** instead of rejecting: delaying or queuing requests, so the work still completes, just later. (Many people and docs use “throttling” loosely to mean rate limiting; say which one you mean.)
- **Load shedding** is driven by the **server's health**, not the client's limit. When the system is overloaded, it drops low-priority work to protect everything else, even if every client is within its limits.

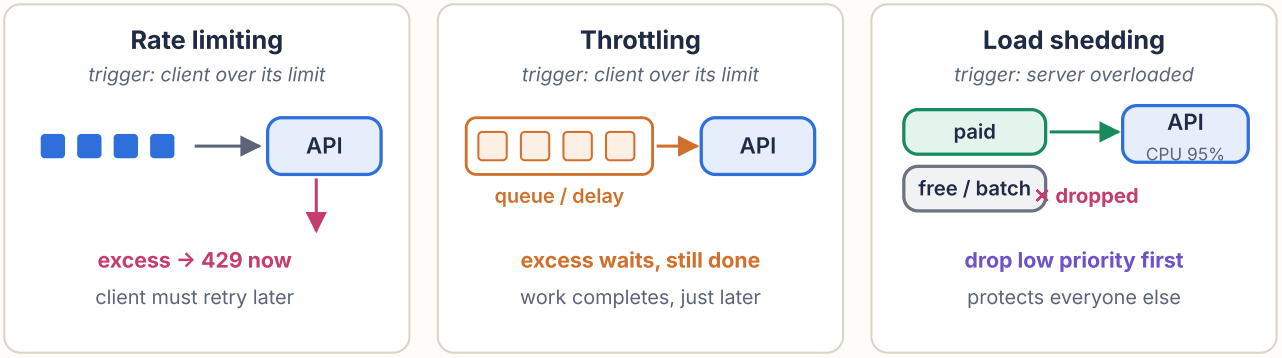


Figure 1. Same excess traffic, three responses. The first two are about one client's limit; load shedding is about the server's own health.

3 Quotas: long-term budgets tied to billing

Quotas are budgets per day or per month, usually tied to the customer's plan and invoice. Because they cost money, they need **durable, accurate** counting. A lost counter is not just a glitch: it is free usage you can't bill, or a customer blocked by mistake.

	Rate limit	Quota
Time scale	Seconds to minutes	Days to months
Purpose	Protect the system from bursts	Enforce the business plan
Counting	Fast, approximate is OK, losing state is OK	Durable, exact, must survive failures
When exceeded	429, retry in seconds	429 or 402, or overage billing, until the next period

4 Enforcing a monthly quota

1. Keep a **fast counter** per tenant, per billing period, in Redis (for example `usage:t42:2026-11`), and check it at the gateway on every request.
2. In the background, **stream usage events** to a durable store. That store, not Redis, is the **source of truth for billing**.
3. **Reconcile** the Redis counter from the durable store periodically, so losing a Redis node never resets anyone to zero.
4. **Reset at each tenant's own billing anchor date** (the day they subscribed), not at midnight on the 1st for everyone.

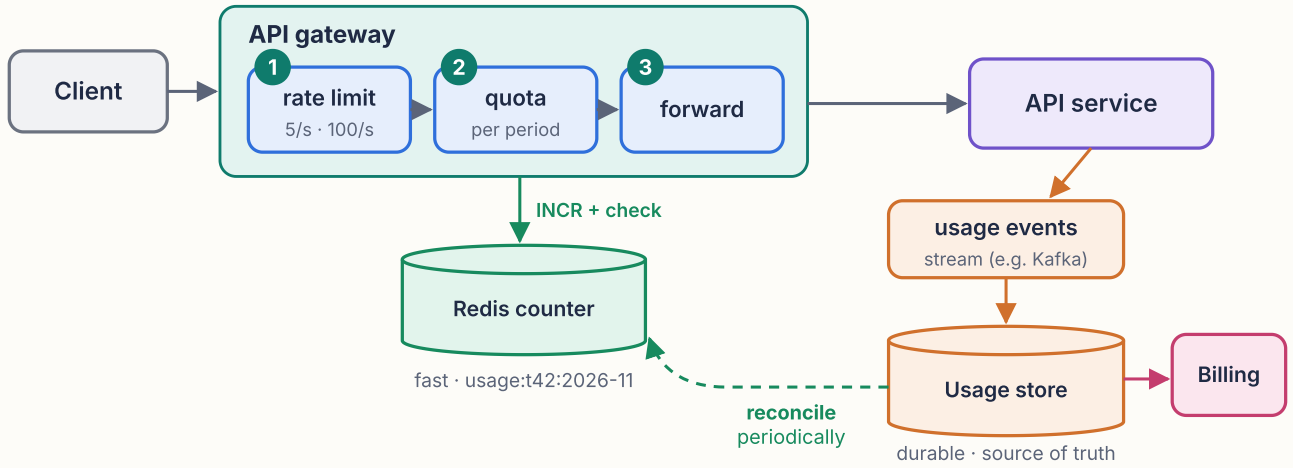


Figure 2. The gateway checks the rate limit (1), then the quota (2), then forwards (3). Redis answers fast; the event stream and usage store keep billing exact.

5 Make limits humane, and fair

Warn at 80% and 100%, by email and in response headers, so nobody is surprised. Then choose per plan:

- **Hard limit:** return 429, or `402 Payment Required`, with a link to upgrade. Right for the free tier.
- **Soft limit:** let paying customers go over and bill the overage. Right for paid tiers. **Never cut off a paying customer silently.**

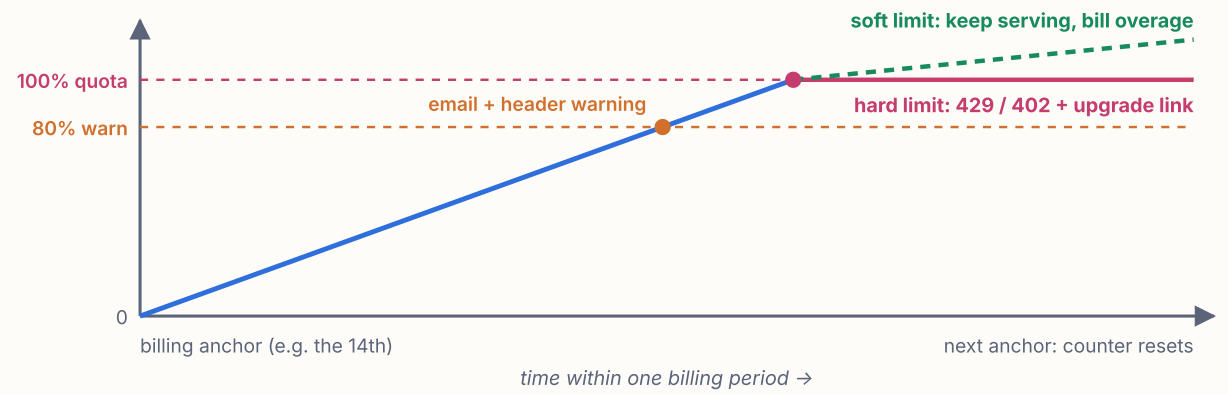


Figure 3. One billing period. A warning at 80%, then a hard stop (free tier) or continued service with overage billing (paid tier).

Tiers also decide fairness under pressure. Give each tier its own rate, burst and quota. Put paid traffic in a separate queue or worker pool, so a noisy free user can never starve paying customers. And when the system is overloaded, shed background jobs and the free tier first.

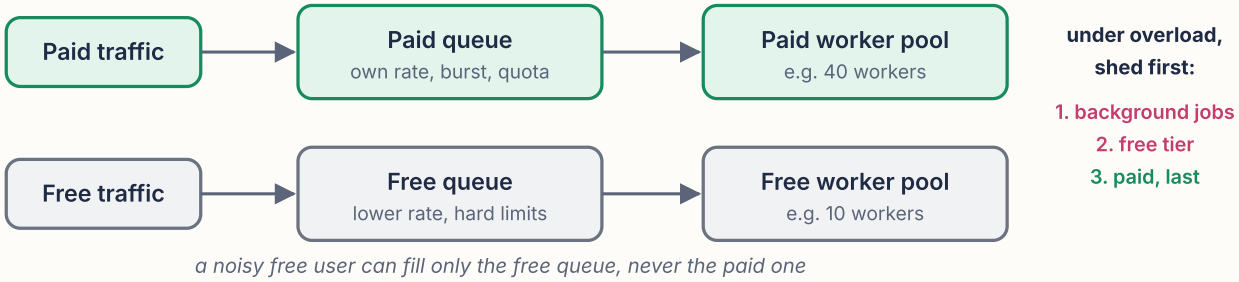


Figure 4. Isolating tiers. Separate queues and worker pools mean one tier's spike can't use up another tier's capacity.

WORKED EXAMPLE: PUTTING IT TOGETHER

	Free tier	Paid tier
Quota	1,000 requests / day	1,000,000 requests / month
Rate limit	5 / second	100 / second, with bursts
Over quota	Hard limit: 429 + upgrade link	Soft limit: overage billing

The free user's loop: at 5 per second, 1,000 requests take about 3 minutes 20 seconds, then every call gets 429 until tomorrow. The rate limit slowed them down; the quota stopped them. The paid customer's month-end batch: at 100 per second, 1,000,000 requests take under 3 hours of full-speed use. They get warnings at 800,000 and 1,000,000, then keep going and pay for the overage.

COMMON MISTAKES

- Keeping quota counts only in Redis: a node failure resets customers to zero, or loses billable usage.
- Resetting every tenant's quota on the 1st instead of their own billing anchor.
- Cutting off a paying customer with a bare 429 and no warning.
- Running all tiers through one shared queue, so free traffic starves paid traffic.

6 Key terms

Rate limiting: reject above a short-term rate (429)

Throttling: slow or queue instead of reject

Load shedding: drop low-priority work when overloaded

Quota: long-term budget per plan

Billing anchor: the tenant's own period start

Reconcile: correct the fast counter from the durable store

Hard limit: stop at the quota

Soft limit: go over, bill overage

402: Payment Required

7 Interview questions

Q1. How do throttling and rate limiting differ? BASIC

Rate limiting rejects requests above a short-term rate with 429. Throttling slows or queues them, so the work still completes, later. Load shedding is a third thing: dropping low-priority work based on server health.

Q2. How are quotas different from rate limits? BASIC

Rate limits protect the system over seconds and can tolerate approximate counting. Quotas enforce the business plan over days or months, drive billing, and need durable, exact counts.

Q3. How do you enforce a monthly quota of 1,000,000 requests? SCENARIO

A fast per-tenant, per-period counter in Redis checked at the gateway; usage events streamed to a durable store as the billing source of truth; periodic reconciliation of Redis from it; reset at each tenant's billing anchor; warnings at 80% and 100%; a hard or soft limit per plan.

Q4. A Redis node holding quota counters fails. What happens to customers? SCENARIO

With reconciliation, the counter is rebuilt from the durable usage store, so nobody resets to zero. Usage during the gap is still recorded in the event stream, so billing stays exact; at worst a tenant goes a little over their quota before the counter catches up.

Q5. Free users are slowing down paying customers during spikes. Fix? SCENARIO

Isolate tiers: separate queues or worker pools and per-tier rate limits, so free traffic can only fill its own capacity. Under overload, shed background jobs and the free tier first.

Q6. A free user is at 5 requests per second. Why do they still need a daily quota? TRICKY

5 per second is 432,000 per day. Without a quota, a loop running all day stays within the rate limit but uses 432 times the free allowance of 1,000. The rate limit shapes speed; the quota caps total consumption.

Q7. When would you return 402 instead of 429? TRICKY

429 says “too fast, retry later”, which suits rate limits. When the monthly quota is exhausted, waiting a few seconds won't help; 402 Payment Required (with an upgrade link) tells the client the fix is a plan change. Some APIs use 429 for both, with a clear error code.

Next: SD-C-015 · API Authentication backendroadmap.com